210002191

CS3105 Practical 1 Report

Overview

In this practical we were tasked to create a program that could verify solutions for a generalized version of Late Binding Solitaire, both according to its strict ruleset as well as according to a more relaxed "Saving Grace" version. This task was achieved. We were also tasked to implement solving ability for both versions of the game, with the use of Dept-First Search, and this was also mostly achieved.

Design

The starting code provided by Ian Gent was excellent for the purpose of parsing input, hence practically no change was required to be made to it. Since most of the code required for checking or solve non-grace versus grace puzzles was the same, two helper functions were created for each task, `check()` and `solve()`.

The check method takes in the layout of the puzzle, as well as the list of moves of the solution that needs to be evaluated. It also takes in a flag indicating whether a saving grace move is allowed. The actual operation of checking a solution is straightforward. Preliminary checks are conducted on both the puzzle and the solution to ensure they adhere to the set standard. We then create a "live" card set-up called play which is used to run each of the moves in the solution step-by-step. If an illegal move is detected, we can produce false as output. If we manage to run all the moves and reach a solved state – i.e., a single pile left – we then produce true as output. This obviously has O(n) time complexity.

For Saving Grace to matter in this case, two conditions need to apply (other than the grace flag being set to true). Firstly, the move must violate either the rule about movements being 1-pile or 3-pile, or the rule about being the same suit/value. Secondly, the grace must not have been carried out yet. Hence, when the first condition is hit, a check for three booleans, namely that grace is allowed, that grace has not been done yet and that the move is indeed towards the zeroth pile, is carried out, after which we record that a grace has been carried out and we carry on compiling the moves.

The solve method is obviously a bit more of an involved process. While it has fewer inputs (no solution to look at), it cannot simply carry out a linear iteration with stop conditions the way the check method does. Instead, it makes use of a custom data structure, a Tree, to find a solution to the given puzzle. Each Tree is constructed using the current layout of cards, the number of ranks for that particular puzzle, as well as a flag indicating whether a grace move is possible.

The `dfSearch()` method is the main trigger for the act of finding a solution to a given puzzle. However this is mainly a public wrapper which calls the private `search()` method and parses the obtained search outcome into the specified output format.

The `search()` method uses recursion to explore the move space and find a solution (or lack thereof). First and foremost, there are two base cases. If there is only a single pile, the puzzle is already solved. The other base case is that the entire search operation has taken longer than the given time-limit (this is why the search() method takes an initial time as a parameter). If neither of these have been reached, we proceed to the actual search operation. The steps necessary have been separated where possible into a few different methods. Firstly, the grow method expands

the move space by creating trees for all the possible future board layouts and adding them as children to the current tree. We then sort the children according to a metric known as “entropy”, which is essentially the density of future moves of a certain layout. Since all valid moves decrease the number of piles by 1, a layout with higher entropy (i.e. more future children) has a higher probability of being the tree that eventually leads to a solution state, i.e. a single pile. We then check each child in turn, and if they are not themselves a solution state (which would finish the search) or of zero entropy, we carry out the recursive step of carrying out a search on that child until a solution is eventually found.

Testing

The stacscheck tests were rigorous and extensive. Hence, they were determined to be sufficient to ascertain the functionality of the program. It was found that full test pass rate was obtained in the first two parts, however, due to time-outs, part three had a success rate of $\frac{65}{67}$. We also note that checking is a very fast process that takes an average of 123ms per input. Solving on the other hand has a noticeable time footprint, with an average of 6570ms per input. The additional states generated by the depth search tree with the Saving Grace mechanic do not appear to significantly alter the time footprint with an average of 6061ms per input.

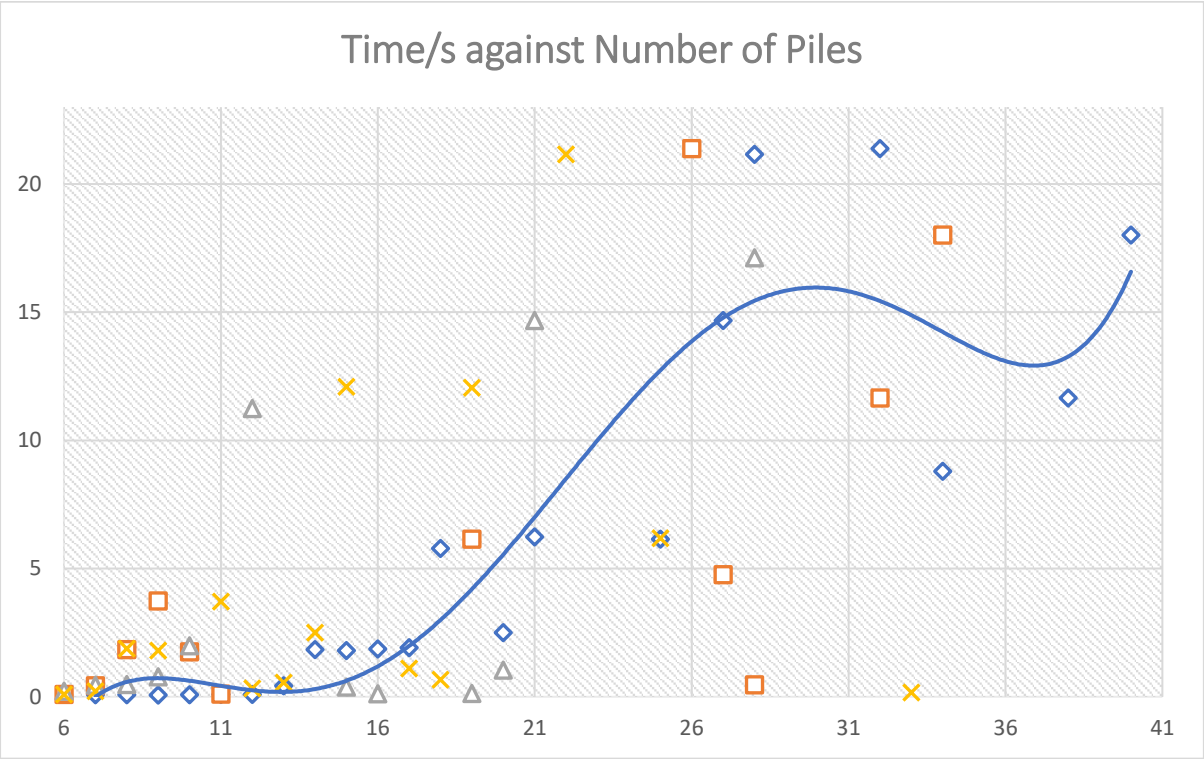
```
[redacted]@palain:/cs/home/djn9/nginx_default/AI-1 $ time stacscheck /cs/home/[redacted]/nginx_default/AI-1/Tests/
Part1
Testing CS3105 P1
- Looking for submission in a directory called 'src': found at 'StarterCode/src'
* BUILD TEST - build-all : pass
* COMPARISON TEST - Part1/prog-run-01Correct.out : pass
* COMPARISON TEST - Part1/prog-run-02Wrong.out : pass
* COMPARISON TEST - Part1/prog-run-03Correct.out : pass
* COMPARISON TEST - Part1/prog-run-04Wrong.out : pass
* COMPARISON TEST - Part1/Correct/prog-run-seed10.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/Correct/prog-run-seed11.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/Correct/prog-run-seed12.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/Correct/prog-run-seed13.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/Correct/prog-run-seed14.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/Correct/prog-run-seed15.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/Correct/prog-run-seed16.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/Correct/prog-run-seed17.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/Correct/prog-run-seed18.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/Correct/prog-run-seed19.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/GraceClones/prog-run-seed10.p12.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/GraceClones/prog-run-seed11.p12.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/GraceClones/prog-run-seed12.p12.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/GraceClones/prog-run-seed12.p16.r10.s4.txt.out : pass
* COMPARISON TEST - Part1/GraceClones/prog-run-seed13.p12.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/GraceClones/prog-run-seed13.p16.r10.s4.txt.out : pass
* COMPARISON TEST - Part1/GraceClones/prog-run-seed14.p12.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/GraceClones/prog-run-seed14.p16.r10.s4.txt.out : pass
* COMPARISON TEST - Part1/GraceClones/prog-run-seed15.p12.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/GraceClones/prog-run-seed16.p12.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/GraceClones/prog-run-seed16.p16.r10.s4.txt.out : pass
* COMPARISON TEST - Part1/GraceClones/prog-run-seed17.p12.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/GraceClones/prog-run-seed18.p12.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/GraceClones/prog-run-seed19.p12.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/GraceClones/prog-run-seed20.p12.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/Wrong/prog-run-seed10.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/Wrong/prog-run-seed11.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/Wrong/prog-run-seed12.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/Wrong/prog-run-seed13.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/Wrong/prog-run-seed14.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/Wrong/prog-run-seed15.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/Wrong/prog-run-seed16.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/Wrong/prog-run-seed17.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/Wrong/prog-run-seed18.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part1/Wrong/prog-run-seed19.p24.r8.s3.txt.out : pass
40 out of 40 tests passed

real    0m5.029s
user    0m4.921s
sys     0m1.235s
```

```
@palain:/cs/home/ /nginx_default/AI-1 $ time stacscheck /cs/home/ /nginx_default/AI-1/Tests/Part2
Testing CS3105 P1
- Looking for submission in a directory called 'src': found at 'StarterCode/src'
* BUILD TEST - build-all : pass
* COMPARISON TEST - Part2/EasyUnsol/prog-run-prob3.txt.out : pass
* COMPARISON TEST - Part2/EasyUnsol/prog-run-seed10.p17.r13.s4.txt.out : pass
* COMPARISON TEST - Part2/EasyUnsol/prog-run-seed13.p17.r13.s4.txt.out : pass
* COMPARISON TEST - Part2/EasyUnsol/prog-run-seed18.p17.r13.s4.txt.out : pass
* COMPARISON TEST - Part2/Sol/prog-run-prob0.txt.out : pass
* COMPARISON TEST - Part2/Sol/prog-run-prob1.txt.out : pass
* COMPARISON TEST - Part2/Sol/prog-run-prob2.txt.out : pass
* COMPARISON TEST - Part2/Sol/prog-run-prob4.txt.out : pass
* COMPARISON TEST - Part2/Sol/prog-run-seed10.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part2/Sol/prog-run-seed11.p17.r13.s4.txt.out : pass
* COMPARISON TEST - Part2/Sol/prog-run-seed11.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part2/Sol/prog-run-seed12.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part2/Sol/prog-run-seed13.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part2/Sol/prog-run-seed14.p17.r13.s4.txt.out : pass
* COMPARISON TEST - Part2/Sol/prog-run-seed14.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part2/Sol/prog-run-seed15.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part2/Sol/prog-run-seed16.p17.r13.s4.txt.out : pass
* COMPARISON TEST - Part2/Sol/prog-run-seed16.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part2/Sol/prog-run-seed17.p17.r13.s4.txt.out : pass
* COMPARISON TEST - Part2/Sol/prog-run-seed17.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part2/Sol/prog-run-seed18.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part2/Sol/prog-run-seed19.p24.r8.s3.txt.out : pass
* COMPARISON TEST - Part2/SolveAndCheck/prog-run-seed11.p17.r13.s4.txt.out : pass
* COMPARISON TEST - Part2/SolveAndCheck/prog-run-seed14.p17.r13.s4.txt.out : pass
* COMPARISON TEST - Part2/SolveAndCheck/prog-run-seed16.p17.r13.s4.txt.out : pass
* COMPARISON TEST - Part2/SolveAndCheck/prog-run-seed17.p17.r13.s4.txt.out : pass
27 out of 27 tests passed

real    0m41.655s
user    2m57.390s
sys     0m54.595s
```

```
@palain:/cs/home/ /nginx_default/AI-1 $ time stacscheck /cs/home/
/nginx_default/AI-1/Tests/Part3
Testing CS3105 P1
- Looking for submission in a directory called 'src': found at
'StarterCode/src'
* BUILD TEST - build-all : pass
* COMPARISON TEST - Part3/EasyUnsol/prog-run-seed10.p16.r10.s4.txt.out :
pass
* COMPARISON TEST - Part3/EasyUnsol/prog-run-seed11.p16.r10.s4.txt.out :
pass
* COMPARISON TEST - Part3/EasyUnsol/prog-run-seed15.p16.r10.s4.txt.out :
pass
* COMPARISON TEST - Part3/EasyUnsol/prog-run-seed17.p16.r10.s4.txt.out :
pass
* COMPARISON TEST - Part3/EasyUnsol/prog-run-seed18.p16.r10.s4.txt.out :
pass
* COMPARISON TEST - Part3/EasyUnsol/prog-run-seed19.p16.r10.s4.txt.out :
pass
* COMPARISON TEST - Part3/NoGraceClones/prog-run-seed10.p24.r8.s3.txt.out :
pass
* COMPARISON TEST - Part3/NoGraceClones/prog-run-seed11.p24.r8.s3.txt.out :
pass
* COMPARISON TEST - Part3/NoGraceClones/prog-run-seed12.p24.r8.s3.txt.out :
pass
* COMPARISON TEST - Part3/NoGraceClones/prog-run-seed13.p24.r8.s3.txt.out :
pass
* COMPARISON TEST - Part3/NoGraceClones/prog-run-seed14.p24.r8.s3.txt.out :
pass
* COMPARISON TEST - Part3/NoGraceClones/prog-run-seed15.p24.r8.s3.txt.out :
pass
* COMPARISON TEST - Part3/NoGraceClones/prog-run-seed16.p24.r8.s3.txt.out :
pass
* COMPARISON TEST - Part3/NoGraceClones/prog-run-seed17.p24.r8.s3.txt.out :
pass
* COMPARISON TEST - Part3/NoGraceClones/prog-run-seed18.p24.r8.s3.txt.out :
pass
* COMPARISON TEST - Part3/NoGraceClones/prog-run-seed19.p24.r8.s3.txt.out :
pass
* COMPARISON TEST - Part3/Sol/prog-run-seed10.p12.r8.s3.txt.out : pass
* COMPARISON TEST - Part3/Sol/prog-run-seed10.p17.r13.s4.txt.out : pass
* COMPARISON TEST - Part3/Sol/prog-run-seed11.p12.r8.s3.txt.out : pass
* COMPARISON TEST - Part3/Sol/prog-run-seed11.p17.r13.s4.txt.out : pass
* COMPARISON TEST - Part3/Sol/prog-run-seed12.p12.r8.s3.txt.out : pass
* COMPARISON TEST - Part3/Sol/prog-run-seed12.p16.r10.s4.txt.out : pass
* COMPARISON TEST - Part3/Sol/prog-run-seed12.p17.r13.s4.txt.out : !! 60
second timeout reached
fail
--- expected output ---
16
--- submission output ---
... Terminating due to timeout
---

* COMPARISON TEST - Part3/Sol/prog-run-seed13.p12.r8.s3.txt.out : pass
* COMPARISON TEST - Part3/Sol/prog-run-seed13.p16.r10.s4.txt.out : pass
* COMPARISON TEST - Part3/Sol/prog-run-seed13.p17.r13.s4.txt.out : pass
* COMPARISON TEST - Part3/Sol/prog-run-seed14.p12.r8.s3.txt.out : pass
* COMPARISON TEST - Part3/Sol/prog-run-seed14.p16.r10.s4.txt.out : pass
* COMPARISON TEST - Part3/Sol/prog-run-seed14.p17.r13.s4.txt.out : pass
* COMPARISON TEST - Part3/Sol/prog-run-seed15.p12.r8.s3.txt.out : pass
* COMPARISON TEST - Part3/Sol/prog-run-seed15.p17.r13.s4.txt.out : pass
* COMPARISON TEST - Part3/Sol/prog-run-seed16.p12.r8.s3.txt.out : pass
* COMPARISON TEST - Part3/Sol/prog-run-seed16.p16.r10.s4.txt.out : pass
* COMPARISON TEST - Part3/Sol/prog-run-seed16.p17.r13.s4.txt.out : pass
* COMPARISON TEST - Part3/Sol/prog-run-seed17.p12.r8.s3.txt.out : pass
* COMPARISON TEST - Part3/Sol/prog-run-seed17.p17.r13.s4.txt.out : pass
* COMPARISON TEST - Part3/Sol/prog-run-seed18.p12.r8.s3.txt.out : pass
```

```
* COMPARISON TEST - Part3/Sol/prog-run-seed18.p17.r13.s4.txt.out : pass
* COMPARISON TEST - Part3/Sol/prog-run-seed19.p12.r8.s3.txt.out : pass
* COMPARISON TEST - Part3/Sol/prog-run-seed19.p17.r13.s4.txt.out : pass
* COMPARISON TEST - Part3/Sol/prog-run-seed20.p12.r8.s3.txt.out : pass
* COMPARISON TEST - Part3/SolveAndCheck/prog-run-seed10.p12.r8.s3.txt.out :
pass
* COMPARISON TEST - Part3/SolveAndCheck/prog-run-seed10.p17.r13.s4.txt.out
: pass
* COMPARISON TEST - Part3/SolveAndCheck/prog-run-seed11.p12.r8.s3.txt.out :
pass
* COMPARISON TEST - Part3/SolveAndCheck/prog-run-seed11.p17.r13.s4.txt.out
: pass
* COMPARISON TEST - Part3/SolveAndCheck/prog-run-seed12.p12.r8.s3.txt.out :
pass
* COMPARISON TEST - Part3/SolveAndCheck/prog-run-seed12.p16.r10.s4.txt.out
: pass
* COMPARISON TEST - Part3/SolveAndCheck/prog-run-seed12.p17.r13.s4.txt.out
: !! 60 second timeout reached
fail
--- expected output ---
true
--- submission output ---
false
... Terminating due to timeout
---

* COMPARISON TEST - Part3/SolveAndCheck/prog-run-seed13.p12.r8.s3.txt.out :
pass
* COMPARISON TEST - Part3/SolveAndCheck/prog-run-seed13.p16.r10.s4.txt.out
: pass
* COMPARISON TEST - Part3/SolveAndCheck/prog-run-seed13.p17.r13.s4.txt.out
: pass
* COMPARISON TEST - Part3/SolveAndCheck/prog-run-seed14.p12.r8.s3.txt.out :
pass
* COMPARISON TEST - Part3/SolveAndCheck/prog-run-seed14.p16.r10.s4.txt.out
: pass
* COMPARISON TEST - Part3/SolveAndCheck/prog-run-seed14.p17.r13.s4.txt.out
: pass
* COMPARISON TEST - Part3/SolveAndCheck/prog-run-seed15.p12.r8.s3.txt.out :
pass
* COMPARISON TEST - Part3/SolveAndCheck/prog-run-seed15.p17.r13.s4.txt.out
: pass
* COMPARISON TEST - Part3/SolveAndCheck/prog-run-seed16.p12.r8.s3.txt.out :
pass
* COMPARISON TEST - Part3/SolveAndCheck/prog-run-seed16.p16.r10.s4.txt.out
: pass
* COMPARISON TEST - Part3/SolveAndCheck/prog-run-seed16.p17.r13.s4.txt.out
: pass
* COMPARISON TEST - Part3/SolveAndCheck/prog-run-seed17.p12.r8.s3.txt.out :
pass
* COMPARISON TEST - Part3/SolveAndCheck/prog-run-seed17.p17.r13.s4.txt.out
: pass
* COMPARISON TEST - Part3/SolveAndCheck/prog-run-seed18.p12.r8.s3.txt.out :
pass
* COMPARISON TEST - Part3/SolveAndCheck/prog-run-seed18.p17.r13.s4.txt.out
: pass
* COMPARISON TEST - Part3/SolveAndCheck/prog-run-seed19.p12.r8.s3.txt.out :
pass
* COMPARISON TEST - Part3/SolveAndCheck/prog-run-seed19.p17.r13.s4.txt.out
: pass
* COMPARISON TEST - Part3/SolveAndCheck/prog-run-seed20.p12.r8.s3.txt.out :
pass
65 out of 67 tests passed

real    3m28.315s
user    6m46.085s
sys     1m27.837s
```

Evaluation



A shell script (eval.sh) was written for easily conducting repeated experiments on the effect of problem size on the efficacy of the program. In this case, we have defined problem size as the number of piles, since this is the metric that has the most direct effect on the size of the search tree. The experiment was repeated three times and a curve of best fit was devised.

As shown above, for problems with les than 16 piles, the time taken mostly stays flat at the 0.08-3.72 range (there is a large degree of variation depending on the individual problem). However, after the piles=16 mark, the amount of time taken shoots up polynomially, and again at the piles=36 mark. It is quite likely that after 50 piles the program will start taking more than a minute, which I would personally define as the limit of reasonable processing time.

The main reason for this drastic increase in processing time is largely due to the tree of possible future states expanding exponentially even as a single pile is added. And it should be noted that all of this was done without saving grace allowed. If saving grace had been enabled, even more branches would be added for each layout and the threshold for reliable solving would be reduced even further.

Attempts were made in the design to mitigate this, for example, a number of the iterative methods use memoization to attempt to reduce the number of repeated steps carried out. Furthermore, the entropy-based sorting done on the children in the tree increase the probability that the first chosen child is the one that eventually leads to a solution. It is possible that a more efficient method of pruning exists. It is also possible that there are more areas in which time complexity could be reduced by reducing repeated actions, as well as space complexity by being more proactive in deleting unneeded data.

An attempt was also made to evaluate how the winnability of Late-Binding Solitaire evolves alongside the amount of cards in the deck (i.e. number of ranks × number of suits). The shell script used to assist with this is entitled `eval1.sh` . Theoretically, it was a simple matter of

counting the number of "-1"s that were generated. However, the in-built layout generator in LBSMain kept producing an unusually high number of solvable puzzles. It was unclear how the situation could be rectified due to limited time, hence no meaningful data could be produced.

I do not believe that winnability and hardness are necessarily related concepts. Winnability is simply the likelihood that puzzles produced within a specific limitation would have at least one solution. I would define hardness from a more qualitative position, namely the chance that an imperfect and average intelligence human player would be able to find a solution for a certain layout. A proxy to measure this could be the number of branches in the tree that lead to a solution. A higher score could be given for trees where "zero solution branches" only emerge near the maximum depth, rather than early on in the game. With some modifications to the actual program code as well as a shell script for assistance, this could be measured.

Conclusion

My solution is able to fulfil the requirements of all of the parts in this practical, as testing shows. We were also able to conduct evaluation on the efficiency of the algorithm using a shell script and obtain definitive trends. The algorithm definitely could be made to be faster, since timeouts still occur in situations when they could be avoided.